Slide 1

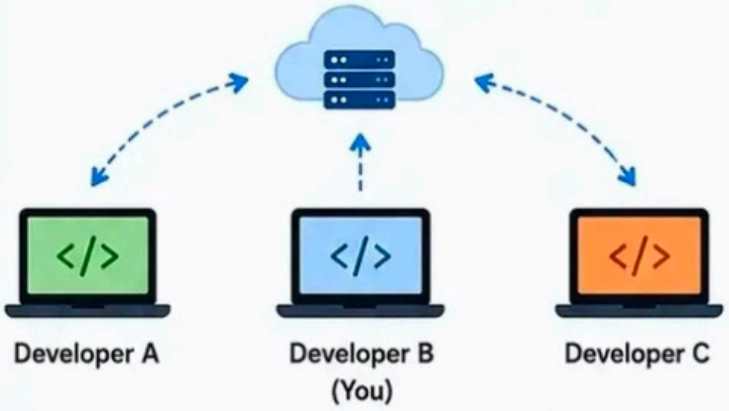
Introduction to Git

Git Cheat Sheet – 20 Essential Commands

What is Git?

Git is a distributed version control system that helps developers track changes in code, collaborate with others, and manage project history efficiently.

How Git Works (Distributed Model)



Common Git Workflow

1. Clone — Get the code from remote repository

2. Modify — Make changes in your local files

3. Add — Stage the changes to be committed

4. Commit — Save the changes with a meaningful message

5. Push — Send changes to remote repository

Goal: Keep your local repository in sync with the remote repository.

Examples

```
# Clone a repository
$ git clone https://github.com/company/project.git
Cloning into 'project'...
remote: Enumerating objects: 120, done.
remote: Total 120 (delta 0), reused 120 (delta 0)
Receiving objects: 100% (120/120), done.

# Navigate to project directory
$ cd project
```

```
# Check status
$ git status
On branch main
Your branch is up to date with 'origin/main'.
nothing to commit, working tree clean

# View commit history
$ git log --oneline
a1b2c3d Update README.md
d4e5f6g Initial commit
```

What Git Helps You Do

- Team Collaboration
- Track Changes
- Maintain History
- Branching & Merging
- Remote Backup & Sync

Slide 2 Repository Setup Commands

1 Initialize Repository

```
git init
```

Creates a new Git repository in the current directory.

MyProject .git

What happens?

- A hidden **.git** folder is created.
- Git starts tracking changes in this folder.

2 Clone Repository

```
git clone https://github.com/
    company/project.git
```

Downloads an existing repository from a remote location.

Remote Repository (GitHub) Local Repository (Your Computer)

What happens?

- Repository is downloaded to your local machine.
- All files, history, and branches are copied.

Slide 3

Tracking Changes

3 Check Status

```
git status
```

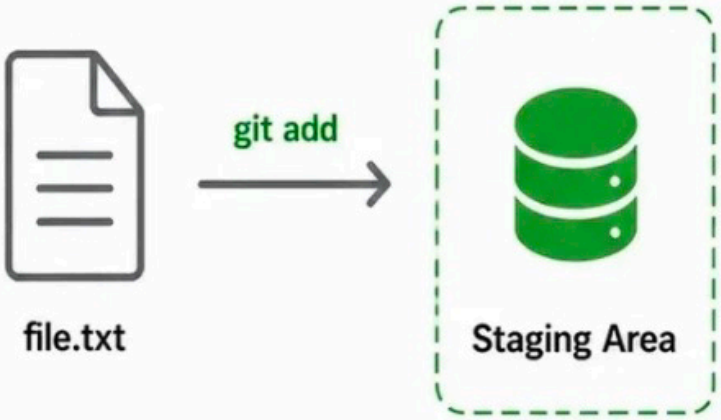
Shows the current state of the working directory and staging area.

```
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update
   what will be committed)
        modified:    app.js
Untracked files:
  (use "git add <file>..." to include
   in what will be committed)
      newfile.txt
no changes added to commit
```

4 Add Changes (Specific File)

```
git add file.txt
```

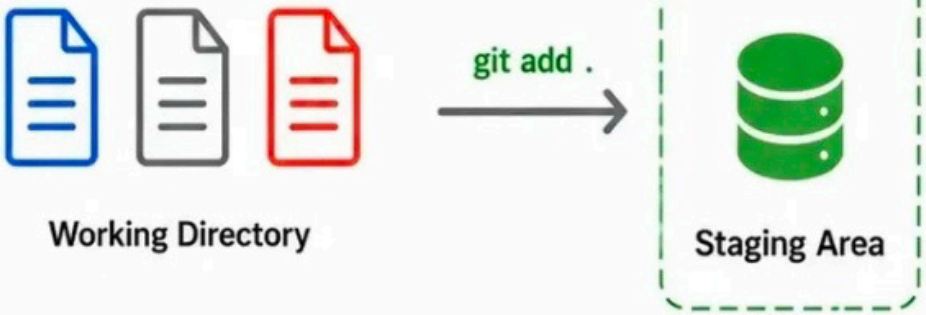
Stages a specific file to be included in the next commit.



5 Add All Changes

```
git add .
```

Stages all changes (new, modified, and deleted files) in the repository.

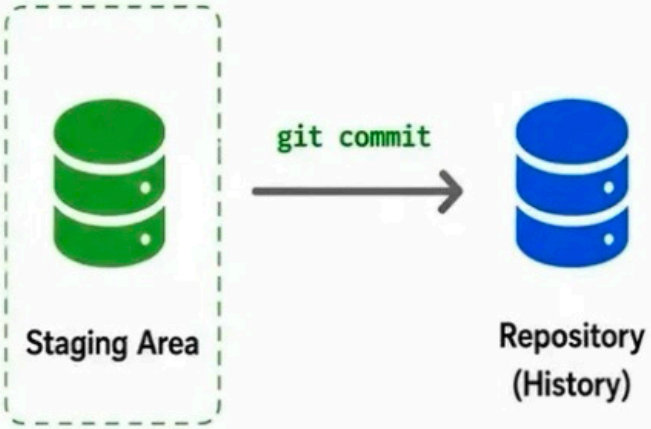


Slide 4 Saving Changes

5 Commit Changes

```
git commit -m "Added login page"
```

Records the staged changes in the repository with a message.



6 View Commit History

```
git log
```

Displays all commits in the repository with full details.

```
commit 8f7a2b1
Author: John Doe
Date:   May 20, 2024
Added login page

commit 4c3d9e8
Author: John Doe
Date:   May 18, 2024
Updated user profile

commit a1b2c3d
Author: John Doe
Date:   May 15, 2024
Initial commit
```

7 Compact History

```
git log --oneline
```

Shows commit history in a compact, single-line format.

Commit ID	Message
8f7a2b1	Added login page
4c3d9e8	Updated user profile
a1b2c3d	Initial commit

Slide 5 Synchronizing with Remote Repository

7 Push Changes

```
git push origin main
```

Uploads your local commits to the remote repository.

8 Pull Latest Changes

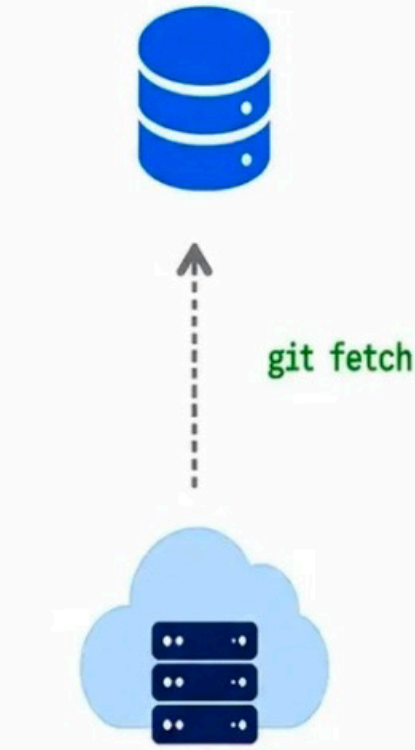
```
git pull origin main
```

Downloads and merges the latest changes from remote to your current branch.

9 Fetch Changes

```
git fetch
```

Downloads the latest changes from remote but does not merge into your branch.



Slide 6 Branch Management

10 View Branches

```
git branch
```

Lists all local branches in the repository.

Branches

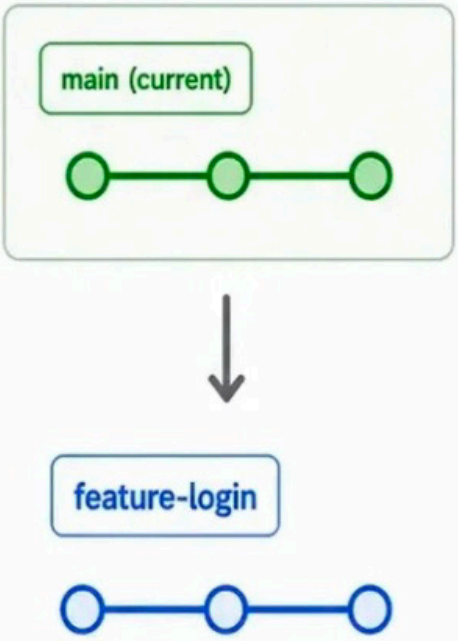
```
* main
  develop
  feature-login
  bugfix-123
```

* indicates the current branch.

11 Create Branch

```
git branch feature-login
```

Creates a new branch with the specified name.



New branch created from current branch.

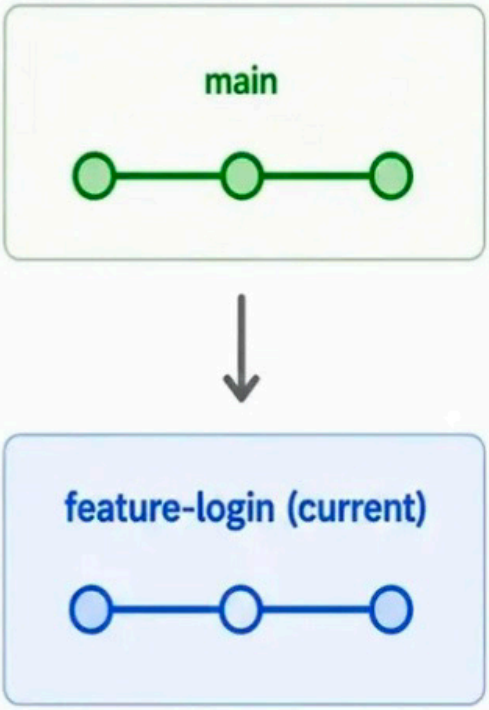
12 Switch Branch

```
git checkout feature-login
```

or

```
git switch feature-login
```

Switches to the specified branch.



You are now on the **feature-login** branch.

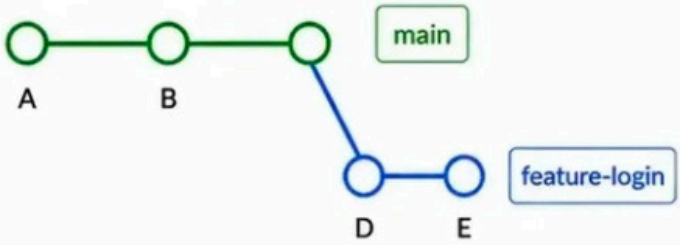
Slide 7 Merging and Rebase

13 Merge Branches

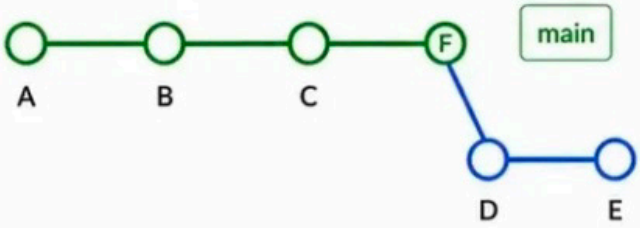
```
git merge feature-login
```

Merges the specified branch into the current branch.

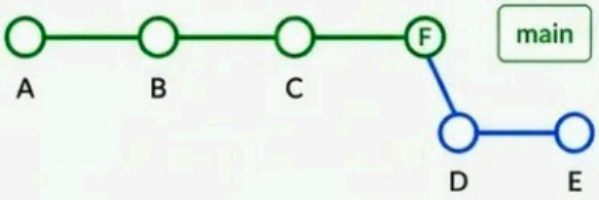
Before Merge



After Merge



Merge creates a new commit (F) that preserves the branch history.



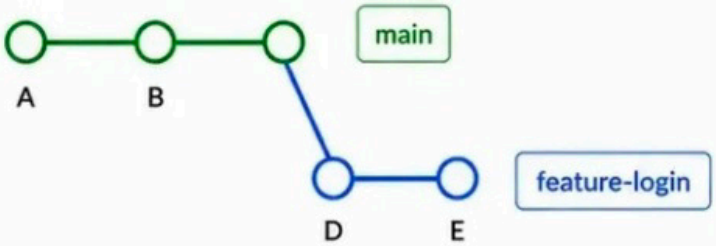
The history remains intact with a merge commit.

14 Rebase Branch

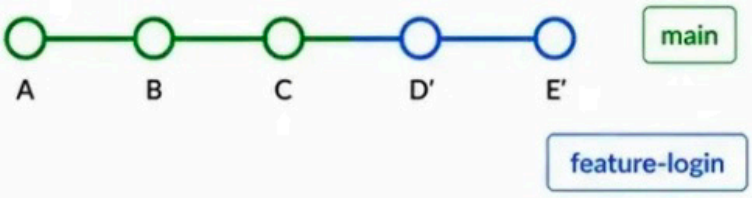
```
git rebase main
```

Reapplies the commits of the current branch on top of the specified branch.

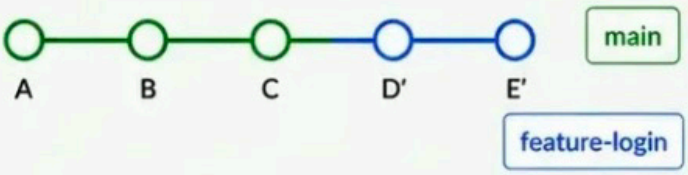
Before Rebase



After Rebase



Rebase moves commits to the top of the target branch.



Creates a cleaner, linear history without merge commits.

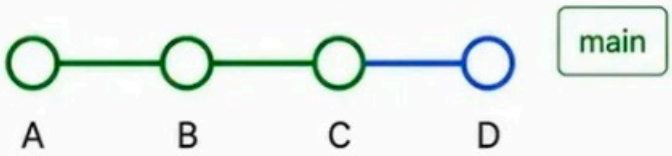
Slide 8 Handling Changes

15 Revert Changes

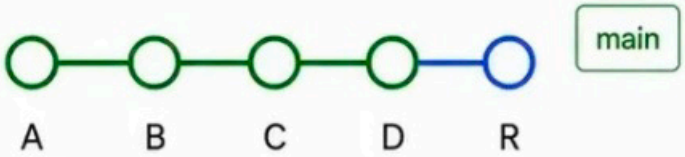
```
git revert <commit-id>
```

Creates a new commit that undoes the changes made in the specified commit.

Before Revert



After Revert



Adds a new commit (R) that reverses the changes of commit C without rewriting history.

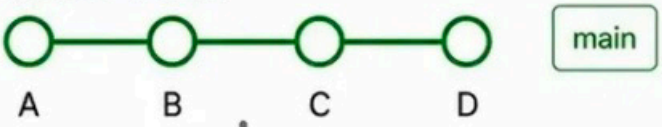
16 Reset Changes

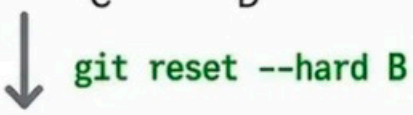
```
git reset <mode> <commit-id>
```

Moves the current branch to a specified commit and updates the working directory and staging area based on the mode.

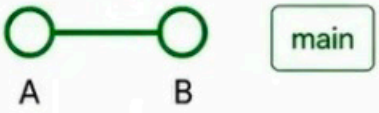
--soft	Moves HEAD only. Changes remain staged.
--mixed (default)	Moves HEAD and unstages changes. Changes remain in working directory.
--hard	Moves HEAD and discards all changes in staging and working directory.

Before Reset





After Reset



Resets the branch to commit B and removes commits C and D from the current branch.

Slide 9 Stashing Changes

17 Stash Changes

```
git stash
```

Temporarily saves your uncommitted changes and reverts your working directory to a clean state.

Before Stash

After Stash

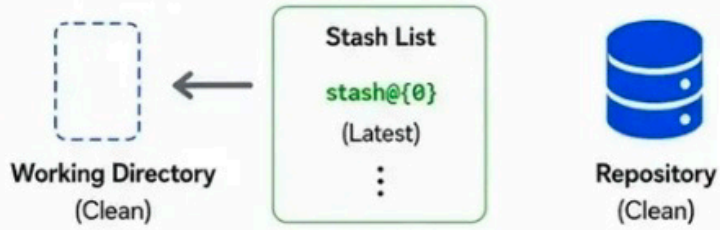
ℹ Changes are saved in the stash list.

18 Apply Stash

```
git stash apply
```

Applies the most recent stash to your working directory without removing it from the stash list.

Before Apply

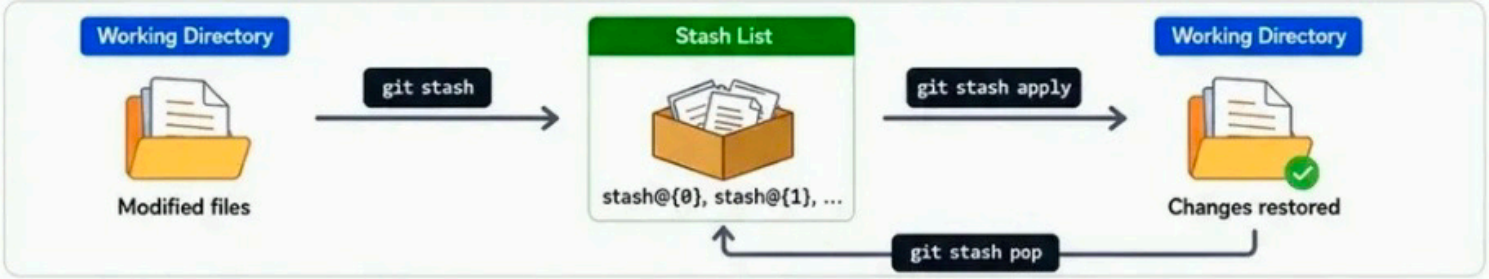


After Apply

ℹ Stash is still available in the stash list.

19 Other Useful Stash Commands

Command	Description
`git stash list`	Lists all stashes.
`git stash pop`	Applies the most recent stash and removes it from the stash list.
`git stash drop stash@{n}`	Deletes the specified stash.
`git stash clear`	Deletes all stashes.



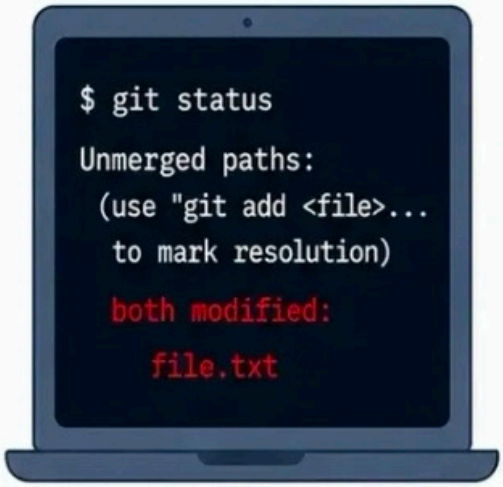
Slide 10

Handling Conflicts

20 Identify Conflicts

```
git status
```

Checks the status to find files with merge conflicts.



file.txt
(Conflict)

21 Resolve Conflicts

```
git (edit files)
```

Open the conflicted files, resolve the conflicts manually, and save the changes.

Conflict in file.txt

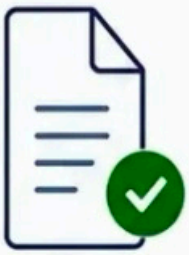
```
<<<<<<< HEAD
Your changes
=======
Incoming changes
>>>>>>> feature-login
```

file.txt
(Resolved)

22 Mark as Resolved

```
git add <file>
```

Stage the resolved files to mark conflicts as resolved.



file.txt
(Resolved)

```
git commit
```

Complete the merge with a commit.